

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: df = pd.read_csv("bank-full.csv", sep=";")
df
```

Out[2]:

	age	job	marital	education	default	balance	housing	loan	contact
0	58	management	married	tertiary	no	2143	yes	no	unknown
1	44	technician	single	secondary	no	29	yes	no	unknown
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown
4	33	unknown	single	unknown	no	1	no	no	unknown
...	...	...	...	...	...	...	...	...	...
45206	51	technician	married	tertiary	no	825	no	no	cellular
45207	71	retired	divorced	primary	no	1729	no	no	cellular
45208	72	retired	married	secondary	no	5715	no	no	cellular
45209	57	blue-collar	married	secondary	no	668	no	no	telephone
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular

45211 rows × 17 columns



Top 10 records

```
In [5]: df.head(10)
```

Out[5]:

	age	job	marital	education	default	balance	housing	loan	contact	day
0	58	management	married	tertiary	no	2143	yes	no	unknown	5
1	44	technician	single	secondary	no	29	yes	no	unknown	5
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5
4	33	unknown	single	unknown	no	1	no	no	unknown	5
5	35	management	married	tertiary	no	231	yes	no	unknown	5
6	28	management	single	tertiary	no	447	yes	yes	unknown	5
7	42	entrepreneur	divorced	tertiary	yes	2	yes	no	unknown	5
8	58	retired	married	primary	no	121	yes	no	unknown	5
9	43	technician	single	secondary	no	593	yes	no	unknown	5

Last 10 records

In [6]: `df.tail(10)`

Out[6]:

	age	job	marital	education	default	balance	housing	loan	contact
45201	53	management	married	tertiary	no	583	no	no	cellular
45202	34	admin.	single	secondary	no	557	no	no	cellular
45203	23	student	single	tertiary	no	113	no	no	cellular
45204	73	retired	married	secondary	no	2850	no	no	cellular
45205	25	technician	single	secondary	no	505	no	yes	cellular
45206	51	technician	married	tertiary	no	825	no	no	cellular
45207	71	retired	divorced	primary	no	1729	no	no	cellular
45208	72	retired	married	secondary	no	5715	no	no	cellular
45209	57	blue-collar	married	secondary	no	668	no	no	telephone
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular

Total number of observations

In [7]: `df.shape` *#total number of row and columns*

Out[7]: (45211, 17)

```
In [8]: print(f"Total rows = {df.shape[0]}")
        print(f"Total columns = {df.shape[1]}")
```

Total rows = 45211

Total columns = 17

Column Names

```
In [9]: df.columns
```

```
Out[9]: Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
              'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
              'previous', 'poutcome', 'y'],
              dtype='object')
```

```
In [10]: df.index
```

```
Out[10]: RangeIndex(start=0, stop=45211, step=1)
```

Datatype of each column

```
In [11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         45211 non-null  int64
 1   job         45211 non-null  object
 2   marital     45211 non-null  object
 3   education   45211 non-null  object
 4   default     45211 non-null  object
 5   balance     45211 non-null  int64
 6   housing     45211 non-null  object
 7   loan        45211 non-null  object
 8   contact     45211 non-null  object
 9   day         45211 non-null  int64
10  month       45211 non-null  object
11  duration    45211 non-null  int64
12  campaign    45211 non-null  int64
13  pdays       45211 non-null  int64
14  previous    45211 non-null  int64
15  poutcome    45211 non-null  object
16  y           45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

Check for any null values

```
In [12]: df.isnull().sum()
```

```
Out[12]: age          0
         job          0
         marital      0
         education    0
         default      0
         balance      0
         housing      0
         loan         0
         contact      0
         day          0
         month        0
         duration     0
         campaign     0
         pdays       0
         previous     0
         poutcome     0
         y            0
         dtype: int64
```

Check for unknown values

```
In [4]: (df=="unknown").sum()
```

```
Out[4]: age          0
         job         288
         marital      0
         education   1857
         default      0
         balance      0
         housing      0
         loan         0
         contact    13020
         day          0
         month        0
         duration     0
         campaign     0
         pdays       0
         previous     0
         poutcome    36959
         y            0
         dtype: int64
```

Check for duplicated records and remove it

```
In [13]: df.duplicated().sum()
```

```
Out[13]: np.int64(0)
```

```
In [25]: df = df.drop_duplicates()
         df.duplicated().sum()
```

```
Out[25]: np.int64(0)
```

Target Variable Distribution

```
In [14]: df['y'].value_counts()
```

```
Out[14]: y
no      39922
yes     5289
Name: count, dtype: int64
```

column analysis

```
In [15]: df.groupby('education')['y'].value_counts()
```

```
Out[15]: education y
primary no      6260
        yes     591
secondary no    20752
          yes    2450
tertiary  no    11305
          yes    1996
unknown  no     1605
         yes     252
Name: count, dtype: int64
```

```
In [16]: df.groupby('marital')['y'].value_counts()
```

```
Out[16]: marital y
divorced no     4585
         yes     622
married  no    24459
         yes    2755
single   no    10878
         yes    1912
Name: count, dtype: int64
```

```
In [17]: df.groupby('contact')['y'].value_counts()
```

```
Out[17]: contact y
cellular no    24916
         yes    4369
telephone no    2516
         yes     390
unknown  no    12490
         yes     530
Name: count, dtype: int64
```

```
In [18]: df.groupby('poutcome')['y'].value_counts()
```

```
Out[18]:
```

poutcome	y	
failure	no	4283
	yes	618
other	no	1533
	yes	307
success	yes	978
	no	533
unknown	no	33573
	yes	3386

Name: count, dtype: int64

```
In [19]: df.groupby('month')['y'].value_counts()
```

```
Out[19]:
```

month	y	
apr	no	2355
	yes	577
aug	no	5559
	yes	688
dec	no	114
	yes	100
feb	no	2208
	yes	441
jan	no	1261
	yes	142
jul	no	6268
	yes	627
jun	no	4795
	yes	546
mar	yes	248
	no	229
may	no	12841
	yes	925
nov	no	3567
	yes	403
oct	no	415
	yes	323
sep	no	310
	yes	269

Name: count, dtype: int64

Summary

```
In [20]: df.describe()
```

Out[20]:

	age	balance	day	duration	campaign	pday
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.00000
mean	40.936210	1362.272058	15.806419	258.163080	2.763841	40.19782
std	10.618762	3044.765829	8.322476	257.527812	3.098021	100.12874
min	18.000000	-8019.000000	1.000000	0.000000	1.000000	-1.00000
25%	33.000000	72.000000	8.000000	103.000000	1.000000	-1.00000
50%	39.000000	448.000000	16.000000	180.000000	2.000000	-1.00000
75%	48.000000	1428.000000	21.000000	319.000000	3.000000	-1.00000
max	95.000000	102127.000000	31.000000	4918.000000	63.000000	871.00000

In [21]: df.describe(include="all")

Out[21]:

	age	job	marital	education	default	balance	housing	loan
count	45211.000000	45211	45211	45211	45211	45211.000000	45211	45211
unique	NaN	12	3	4	2	NaN	2	2
top	NaN	blue-collar	married	secondary	no	NaN	yes	no
freq	NaN	9732	27214	23202	44396	NaN	25130	37967
mean	40.936210	NaN	NaN	NaN	NaN	1362.272058	NaN	NaN
std	10.618762	NaN	NaN	NaN	NaN	3044.765829	NaN	NaN
min	18.000000	NaN	NaN	NaN	NaN	-8019.000000	NaN	NaN
25%	33.000000	NaN	NaN	NaN	NaN	72.000000	NaN	NaN
50%	39.000000	NaN	NaN	NaN	NaN	448.000000	NaN	NaN
75%	48.000000	NaN	NaN	NaN	NaN	1428.000000	NaN	NaN
max	95.000000	NaN	NaN	NaN	NaN	102127.000000	NaN	NaN

In [22]: df.describe(include="object")

Out[22]:

	job	marital	education	default	housing	loan	contact	month	poutcome
count	45211	45211	45211	45211	45211	45211	45211	45211	45211
unique	12	3	4	2	2	2	3	12	4
top	blue-collar	married	secondary	no	yes	no	cellular	may	unknown
freq	9732	27214	23202	44396	25130	37967	29285	13766	36959

Converting column's datatype to numeric

```
In [5]: num_cols = [
        'balance',
        'day',
        'duration',
        'campaign',
        'pdays',
        'previous'
    ]
```

```
In [6]: for col in num_cols:
        df[col] = pd.to_numeric(df[col])

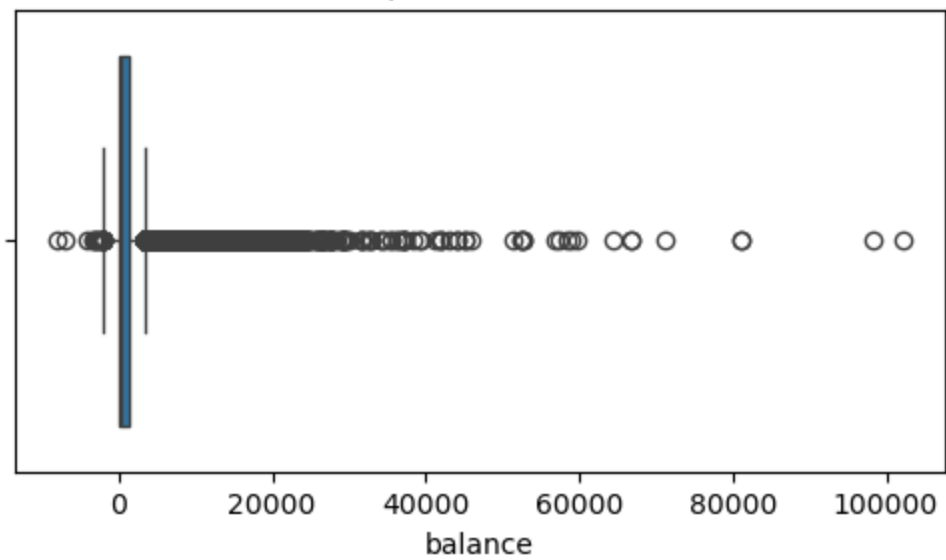
        df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         45211 non-null  int64
1   job         45211 non-null  object
2   marital     45211 non-null  object
3   education   45211 non-null  object
4   default     45211 non-null  object
5   balance     45211 non-null  int64
6   housing     45211 non-null  object
7   loan        45211 non-null  object
8   contact     45211 non-null  object
9   day         45211 non-null  int64
10  month       45211 non-null  object
11  duration    45211 non-null  int64
12  campaign    45211 non-null  int64
13  pdays       45211 non-null  int64
14  previous    45211 non-null  int64
15  poutcome    45211 non-null  object
16  y           45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

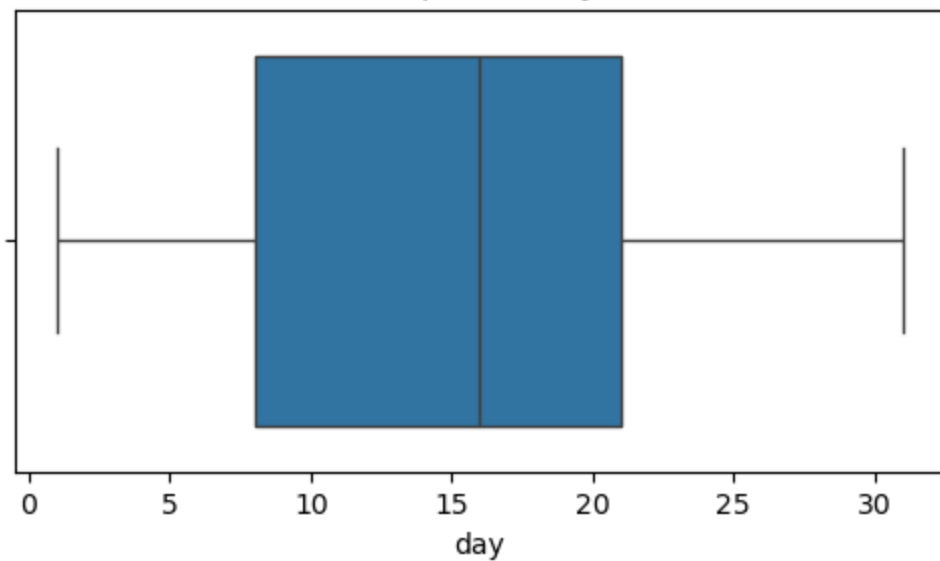
Detect Outliers

```
In [7]: for col in num_cols:
plt.figure(figsize=(6,3))
sns.boxplot(x=df[col])
plt.title(f"Boxplot of {col}")
plt.show()
```

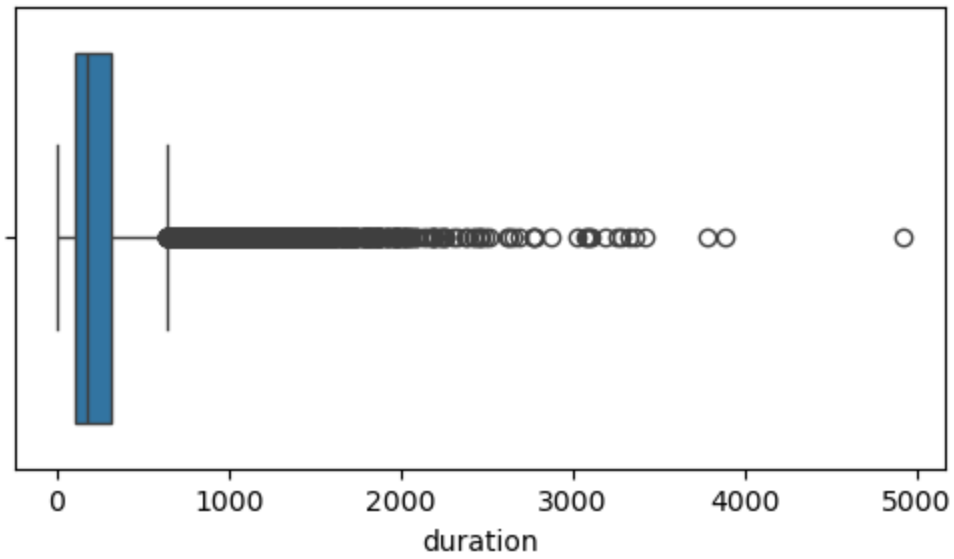
Boxplot of balance



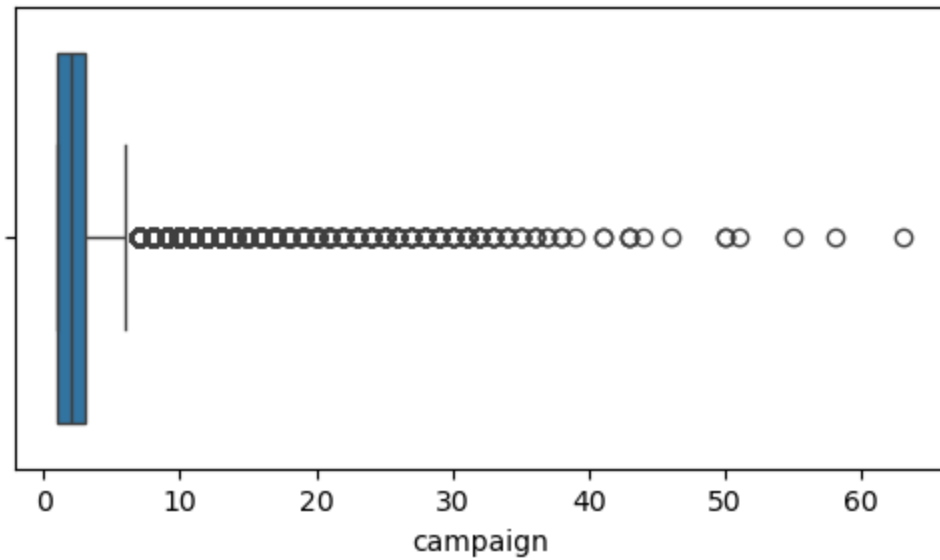
Boxplot of day



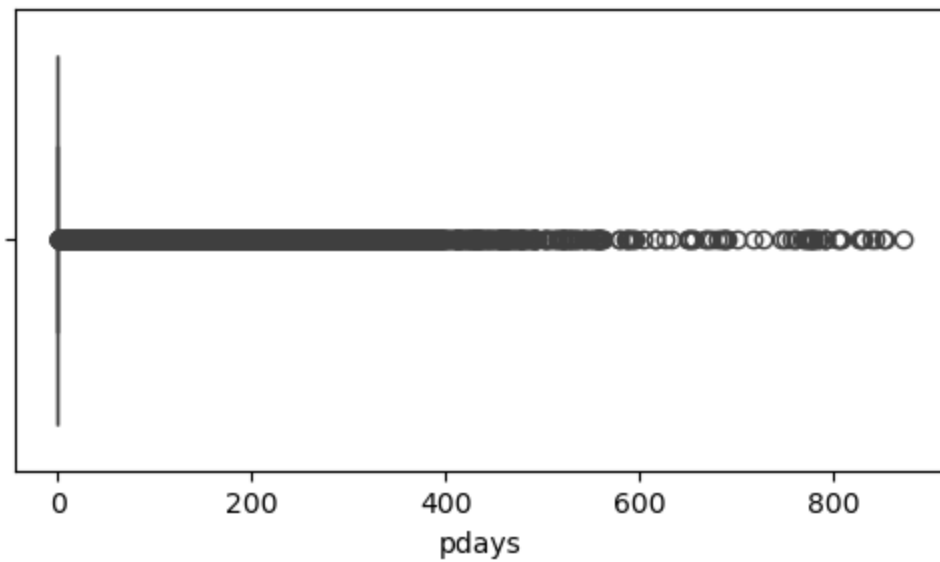
Boxplot of duration

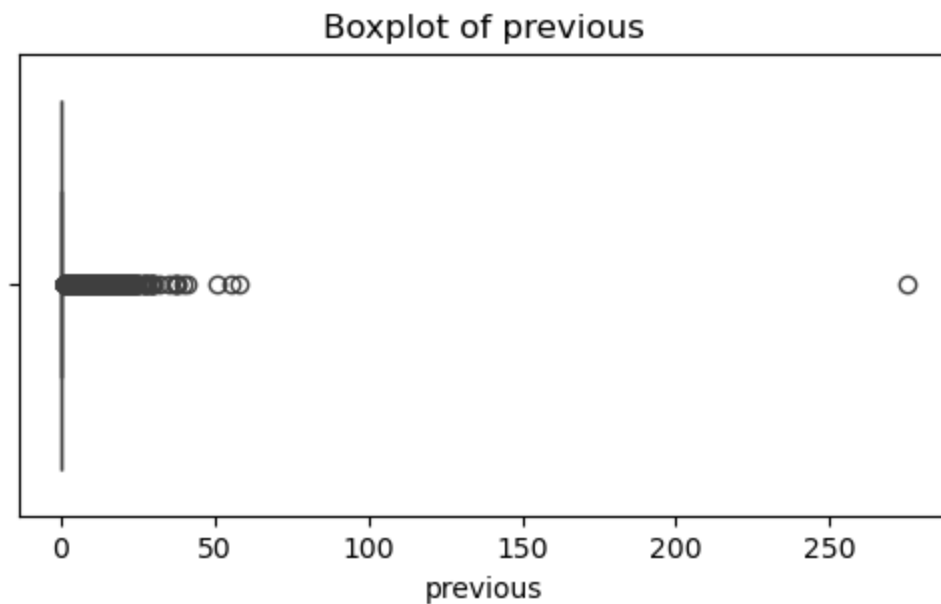


Boxplot of campaign



Boxplot of pdays





Handle Outliers

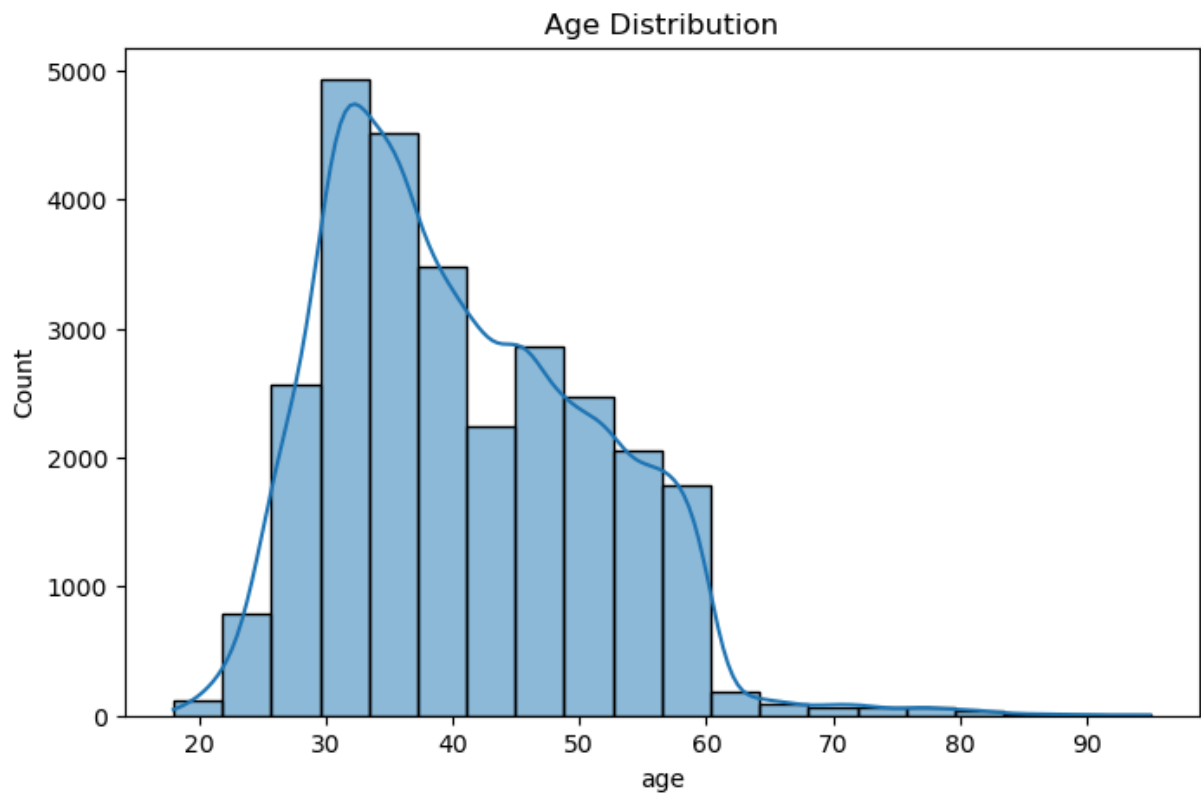
```
In [8]: for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower) & (df[col] <= upper)]
        print(df.shape)
```

(28338, 17)

Exploratory Data Analysis (EDA)

Age Distribution

```
In [9]: plt.figure(figsize=(8,5))
        sns.histplot(df["age"], bins=20, kde=True)
        plt.title("Age Distribution")
        plt.show()
```



Job

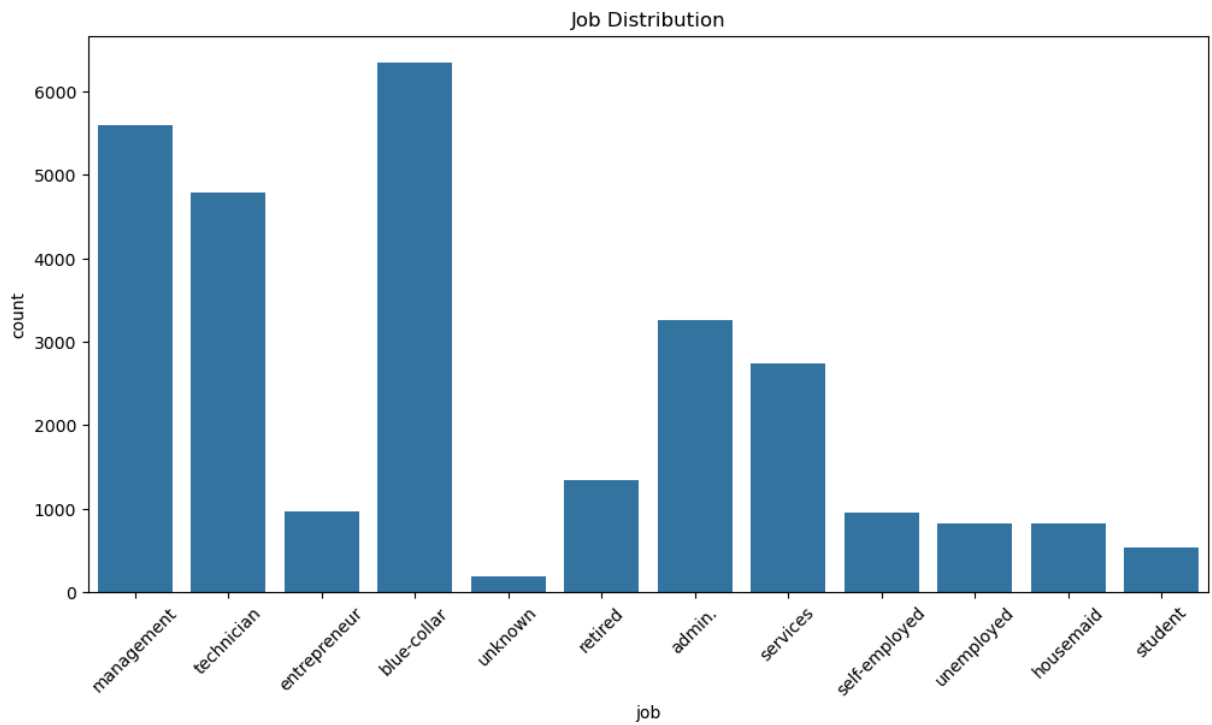
```
In [10]: plt.figure(figsize=(12,6))

sns.countplot(x="job", data=df)

plt.xticks(rotation=45)

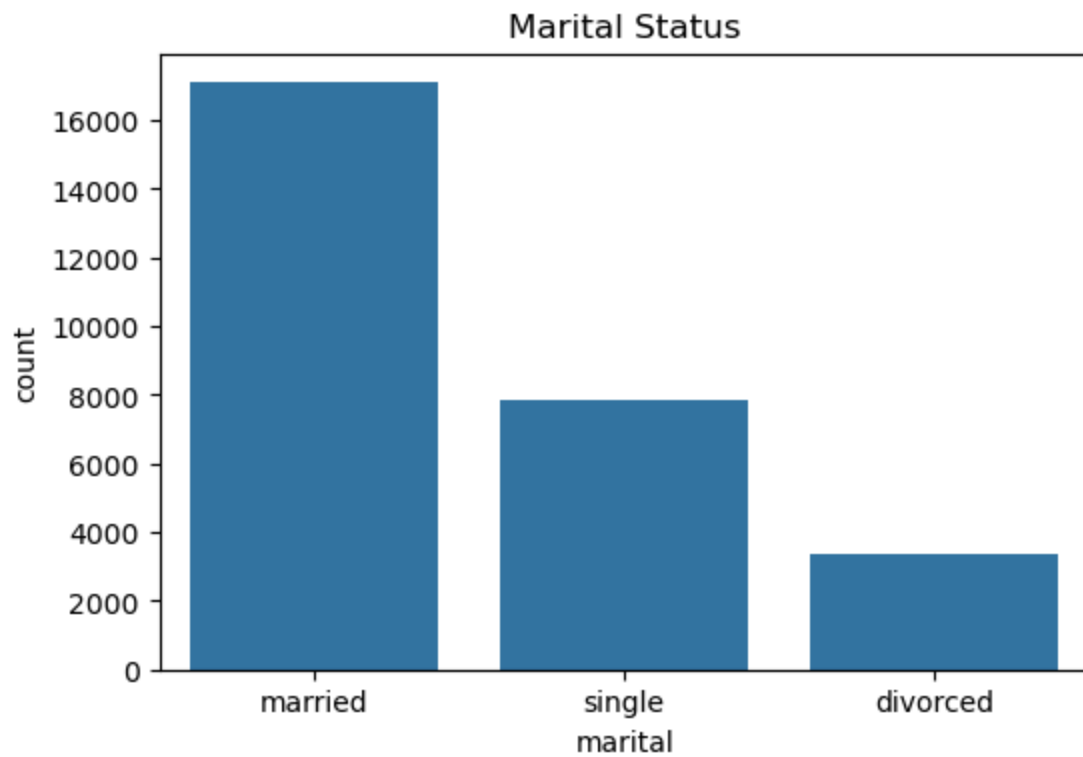
plt.title("Job Distribution")

plt.show()
```



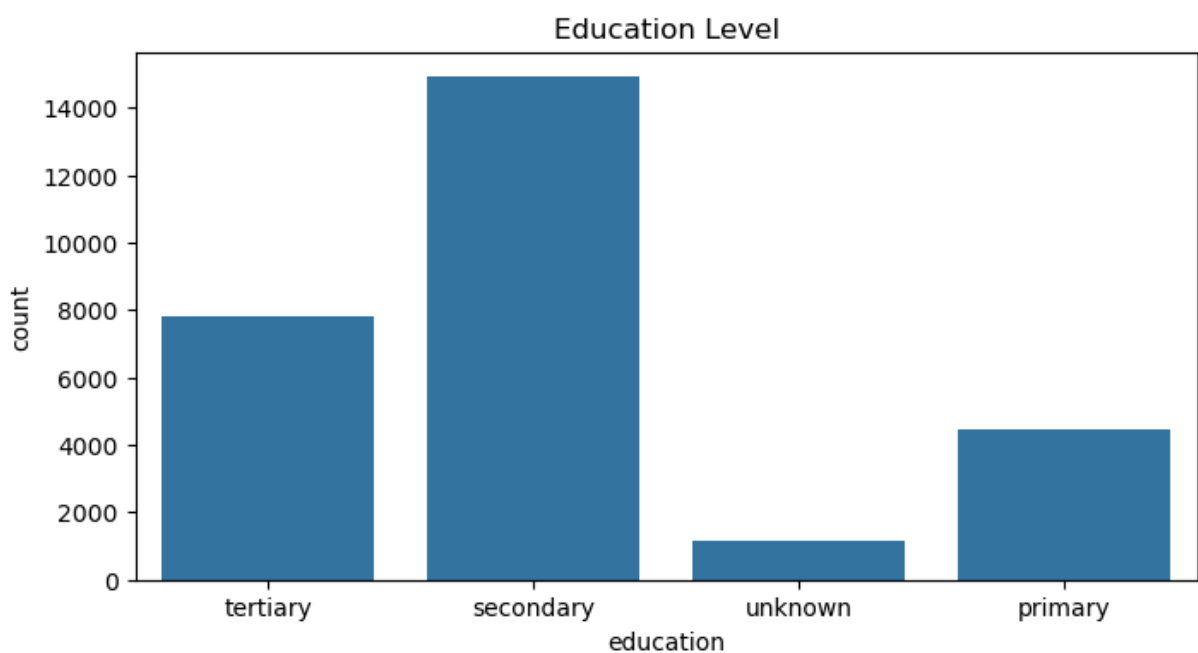
Marital Status

```
In [11]: plt.figure(figsize=(6,4))  
  
sns.countplot(x="marital", data=df)  
  
plt.title("Marital Status")  
  
plt.show()
```



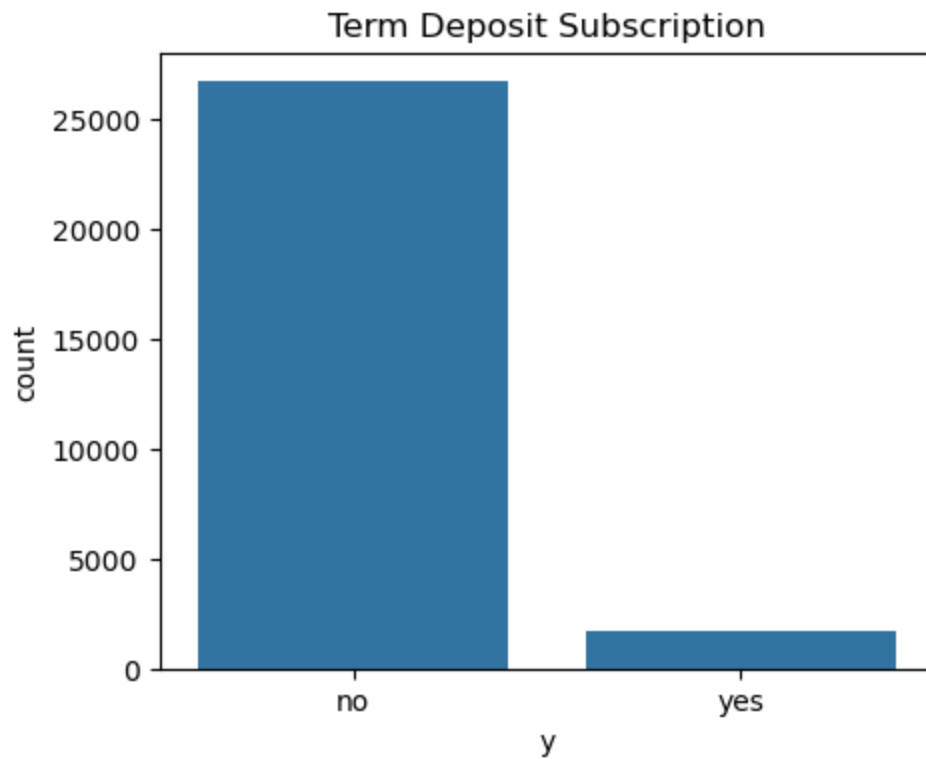
Education Level

```
In [12]: plt.figure(figsize=(8,4))  
sns.countplot(x="education", data=df)  
plt.title("Education Level")  
plt.show()
```



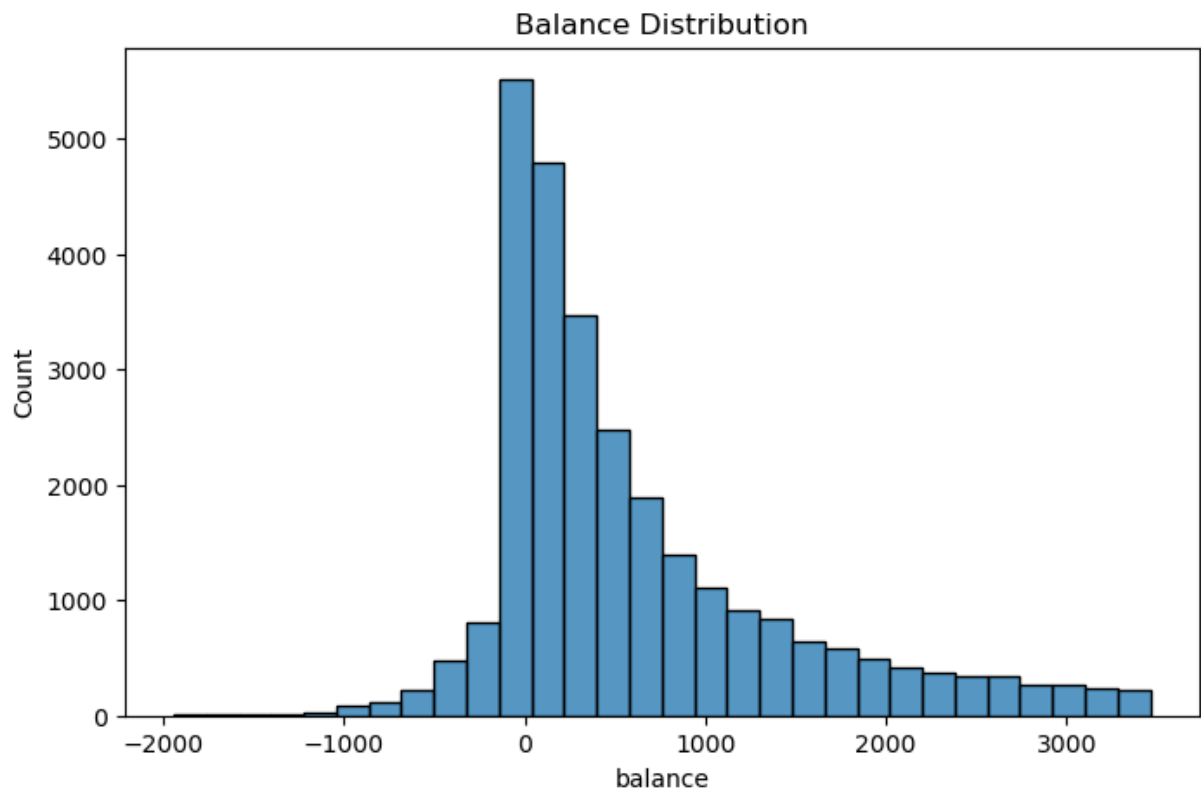
Target Variable

```
In [13]: plt.figure(figsize=(5,4))  
  
sns.countplot(x="y", data=df)  
  
plt.title("Term Deposit Subscription")  
  
plt.show()
```



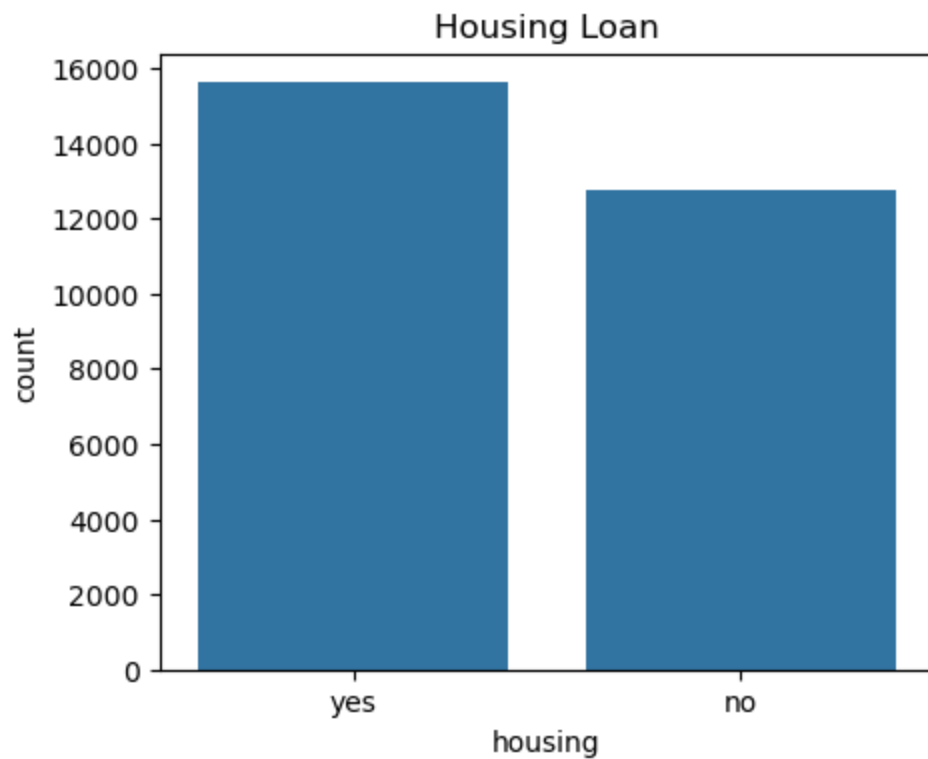
Balance Distribution

```
In [14]: plt.figure(figsize=(8,5))  
  
sns.histplot(df["balance"], bins=30)  
  
plt.title("Balance Distribution")  
  
plt.show()
```



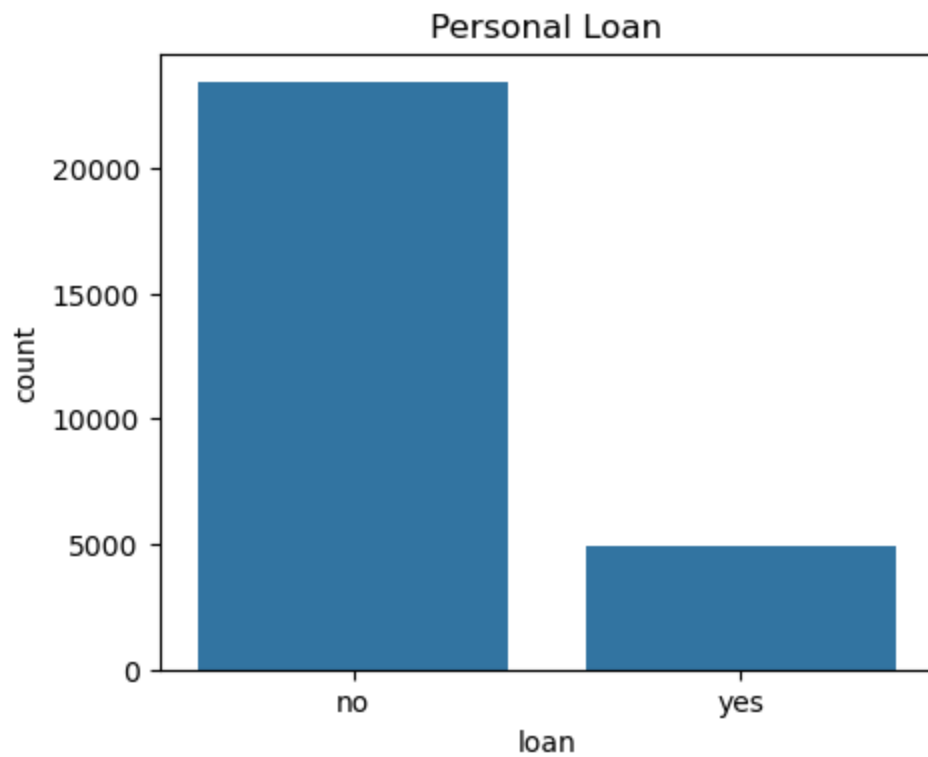
Housing Loan

```
In [15]: plt.figure(figsize=(5,4))  
  
sns.countplot(x="housing", data=df)  
  
plt.title("Housing Loan")  
  
plt.show()
```



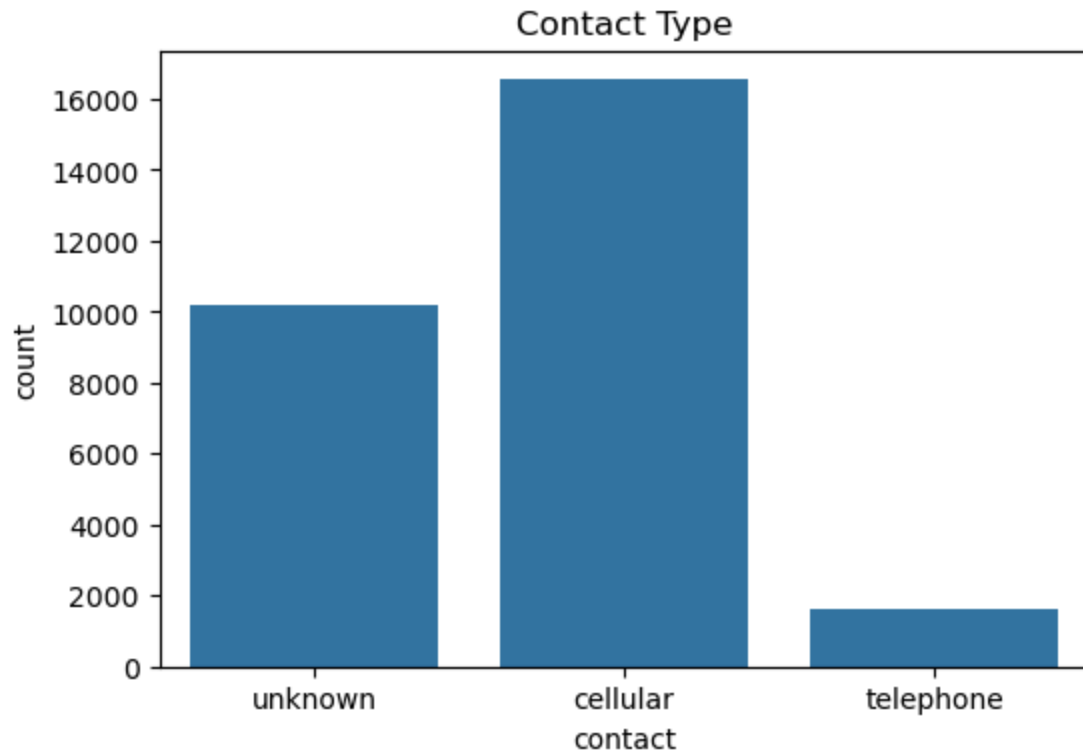
Personal Loan

```
In [16]: plt.figure(figsize=(5,4))  
sns.countplot(x="loan", data=df)  
plt.title("Personal Loan")  
plt.show()
```



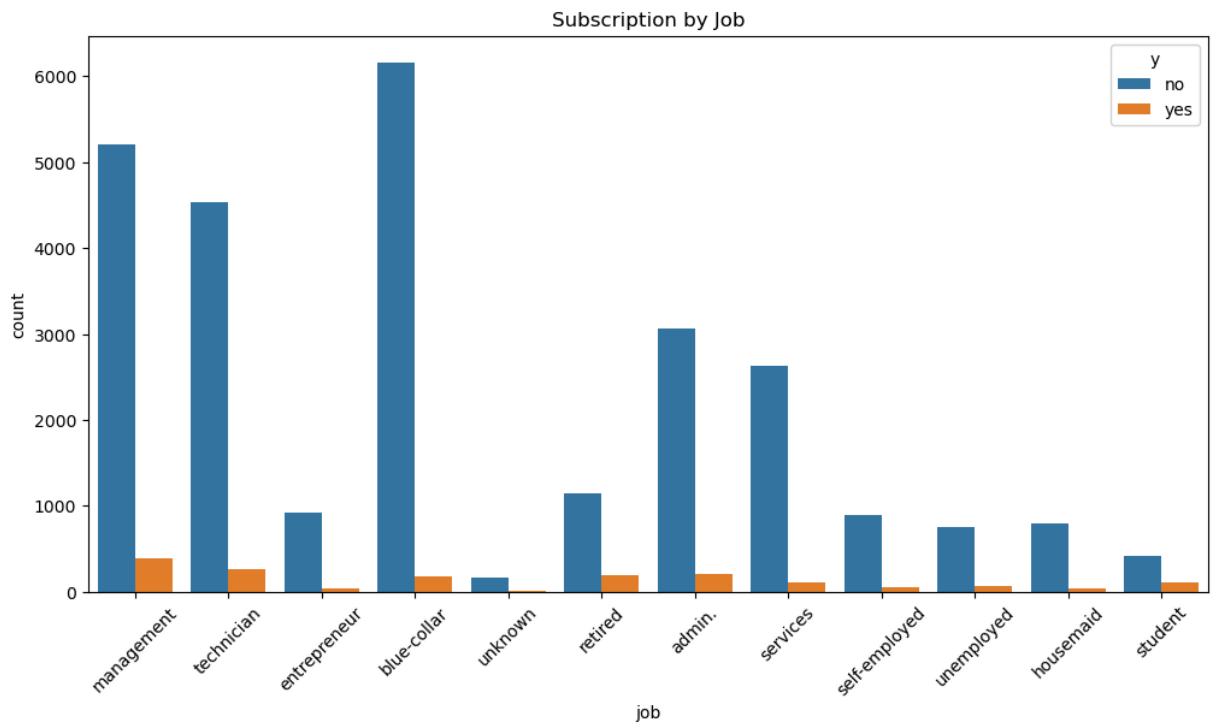
Contact Type

```
In [17]: plt.figure(figsize=(6,4))  
sns.countplot(x="contact", data=df)  
plt.title("Contact Type")  
plt.show()
```



Subscription by Job

```
In [18]: plt.figure(figsize=(12,6))  
  
sns.countplot(x="job", hue="y", data=df)  
  
plt.xticks(rotation=45)  
  
plt.title("Subscription by Job")  
  
plt.show()
```



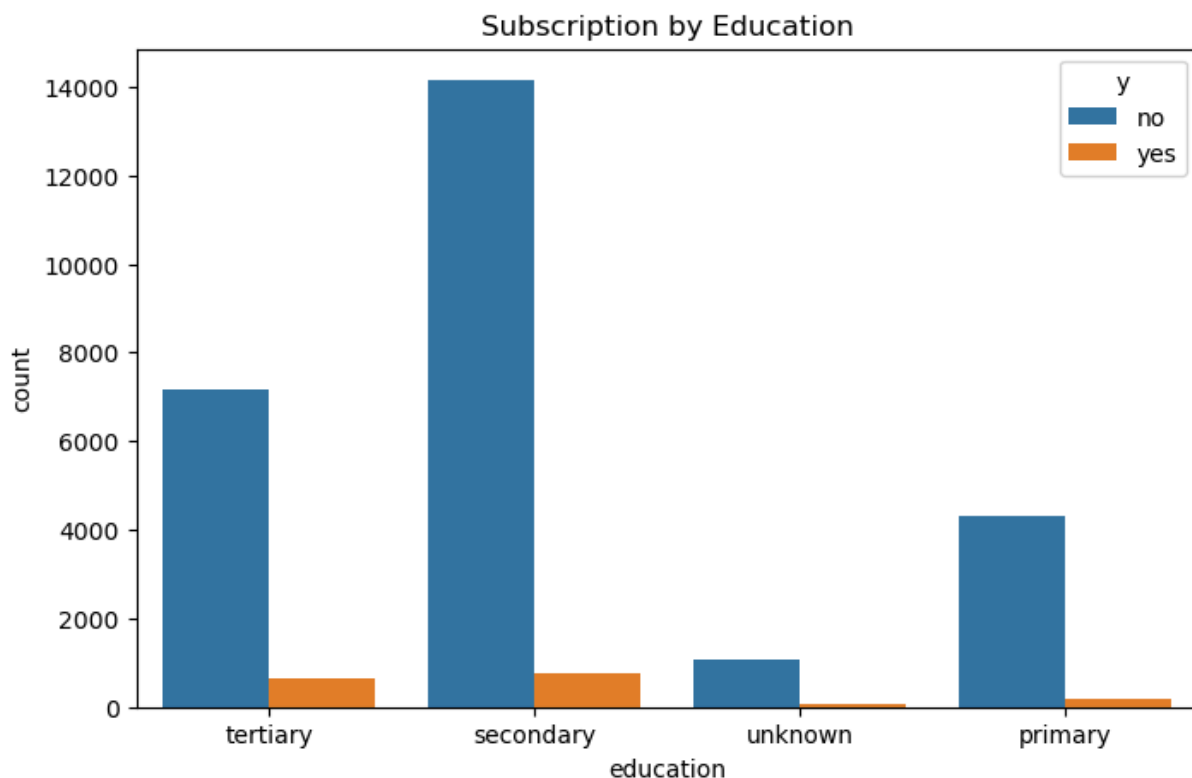
Subscription by Education

```
In [19]: plt.figure(figsize=(8,5))

sns.countplot(x="education", hue="y", data=df)

plt.title("Subscription by Education")

plt.show()
```



Correlation Heatmap

```
In [20]: from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
cat_cols = [  
    'job',  
    'marital',  
    'education',  
    'default',  
    'housing',  
    'loan',  
    'contact',  
    'month',  
    'poutcome',  
    'y'  
]
```

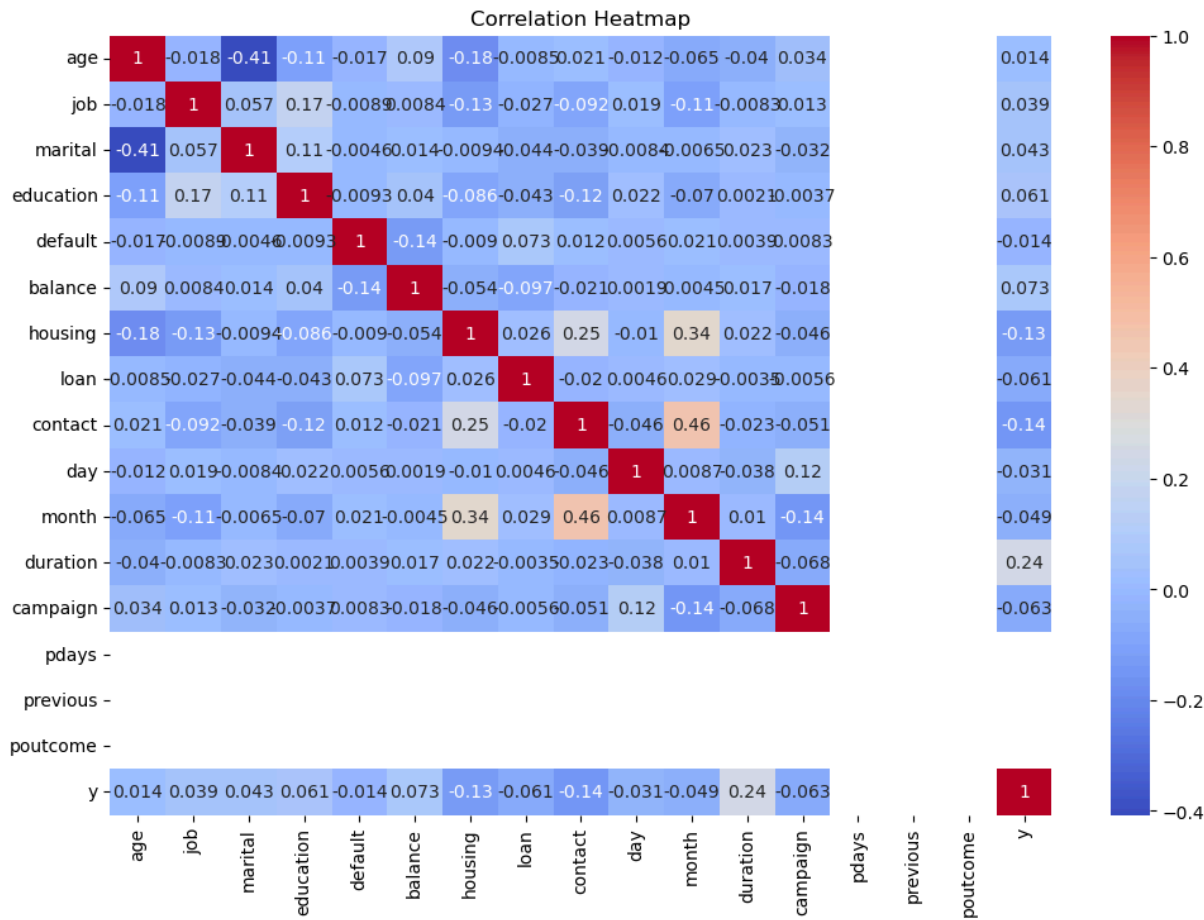
```
for col in cat_cols:  
    df[col] = le.fit_transform(df[col])
```

```
In [21]: plt.figure(figsize=(12,8))
```

```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```



Feature Scaling

```
In [22]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[num_cols] = scaler.fit_transform(df[num_cols])

df.head()
```

Out[22]:

	age	job	marital	education	default	balance	housing	loan	contact	day	mc
0	58	4	1	2	0	1.839343	1	0	2	-1.288418	
1	44	9	2	1	0	-0.696791	1	0	2	-1.288418	
2	33	2	1	1	0	-0.729183	1	1	2	-1.288418	
3	47	1	1	3	0	1.075143	1	0	2	-1.288418	
4	33	11	2	3	0	-0.730382	0	0	2	-1.288418	

Save Cleaned Dataset

```
In [23]: df.to_csv("bank_cleaned.csv", index=False)

print("Cleaned dataset saved successfully.")
```

Cleaned dataset saved successfully.

```
In [ ]:
```